



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-Section SYSTEMS INTEGRATION AND ARCHITECTURE 1

FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

Project Title: WealthWise

Prepared By: Earion Icer O. Bramwell

Date of Submission: 02/06/2026

Version: 1

Table of Contents

- 1. Introduction3
 - 1.1. Purpose.....3
 - 1.2. Scope.....3
 - 1.3. Definitions, Acronyms, and Abbreviations3
- 2. Overall Description3
 - 2.1. System Perspective.....3
 - 2.2. User Classes and Characteristics.....3
 - 2.3. Operating Environment.....4
 - 2.4. Assumptions and Dependencies4
- 3. System Features and Functional Requirements4
 - 3.1. Feature 1:.....4
 - 3.2. Feature 2:.....4
- 4. Non-Functional Requirements.....5
- 5. System Models (Diagrams)6
 - 5.1. ERD.....6
 - 5.2. Use Case Diagram7
 - 5.3. Activity Diagram8
 - 5.4. Class Diagram.....9
 - 5.5. Sequence Diagram10
- 6. Appendices11

1. Introduction

1.1. Purpose

The purpose of WealthWise is to provide students, individuals and freelancers with a simple yet effective platform to track income, expenses, savings, and overall financial health.

1.2. Scope

WealthWise will allow users to register, log in, and manage their personal finance data through a secure dashboard. Users can record transactions, categorize expenses, set budgets, and view analytics. The system boundaries include:

- **In-scope:** User authentication, transaction management, dashboards, reporting.
- **Out-of-scope:** Complex financial forecasting, integration with external banking APIs (future enhancement).

1.3. Definitions, Acronyms, and Abbreviations

- **CRUD:** Create, Read, Update, Delete operations.
- **UI:** User Interface.
- **API:** Application Programming Interface.
- **Transaction:** A financial record of income or expense.
- **Dashboard:** A summary view of financial data and analytics.

2. Overall Description

2.1. System Perspective

WealthWise is a personal finance management web and mobile application. It interacts with a relational database for persistent storage.

2.2. User Classes and Characteristics

Guest User: Can register and log in.

Authenticated User: Can view profile, dashboard, and manage transactions.

2.3. Operating Environment

- Backend: Spring Boot (Java)
- Frontend: ReactJS
- Mobile: Android Kotlin
- Database: PostgreSQL
- Version Control: Git and GitHub
- Deployment: Cloud hosting

2.4. Assumptions and Dependencies

Users have stable internet connection.

Passwords are securely stored using hashing algorithms.

System depends on database availability and uptime.

3. System Features and Functional Requirements

3.1. Feature 1: Expense & Income Logging with Categories

Description: Allows users to record financial transactions like income or expenses and categorize them for better tracking.

Functional Requirements:

- User can add a transaction with fields: amount, category, type (income/expense), date, and notes.
- System must validate that amount is numeric and category is valid.
- User can edit or delete existing transactions.
- Transactions are linked to the authenticated user's account.
- Dashboard updates automatically when new transactions are added.

3.2. Feature 2: Budget Planning

Description: Enables users to set monthly spending limits and receive alerts when nearing or exceeding those limits.

Functional Requirements:

- User can define a monthly budget per category (e.g. food, rent, utilities).
- System tracks total expenses versus the set budget.
- Alerts are triggered when expenses reach 80% of the budget.

- Alerts are triggered again when the budget is exceeded.
- Dashboard displays budget utilization percentage per category.

3.3. Feature 3: Transactional Function (Payments & Bills)

Description: Allows users to record payments and mark bills as paid, ensuring accurate tracking of obligations.

Functional Requirements:

- User can log bill payments with details (amount, due date, status).
- System must allow marking bills as “Paid” or “Unpaid.”
- Paid bills are excluded from pending obligations in the dashboard.
- Transaction history must reflect payment status changes.
- User can filter transactions by status (paid/unpaid).

3.4. Feature 3: Analytical Dashboard

Description: Provides users with visual insights into their financial health through charts and summaries.

Functional Requirements:

- Dashboard displays income vs. expenses in graphical format (pie chart).
- System must calculate monthly savings (income – expenses).
- Dashboard shows spending trends over time (daily, weekly, monthly).
- User can filter analytics by category or date range.
- Charts must update dynamically when new transactions are added.

4. Non-Functional Requirements

Security: Passwords must be hashed. Sessions are managed with tokens. Protected pages cannot be accessed without authentication.

Usability: Intuitive UI with clear navigation; responsive design for mobile and desktop.

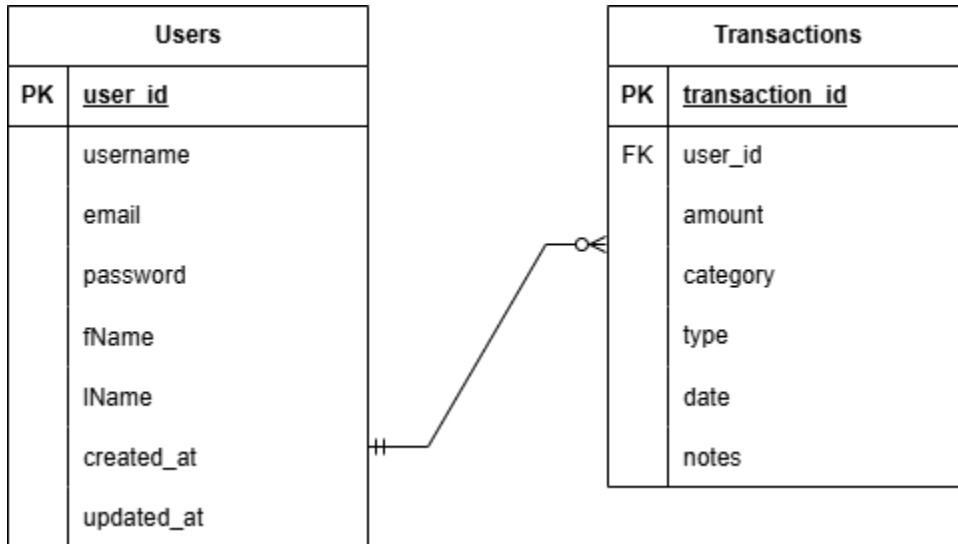
Reliability: System uptime of 99%. Database backups are performed daily.

Scalability: Architecture will allow for future integration with external APIs like banking.

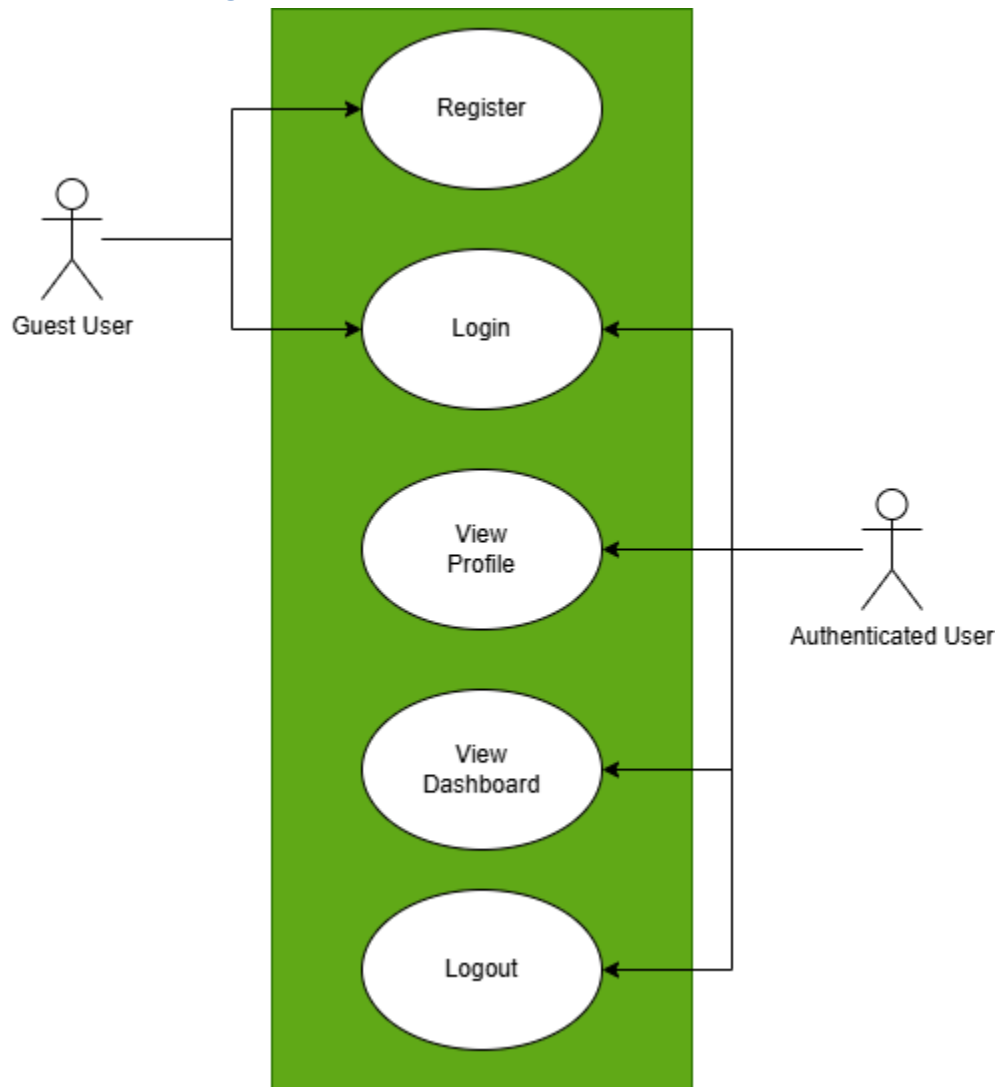
Maintainability: The codebase will be modularly designed for easy updates.

5. System Models (Diagrams)

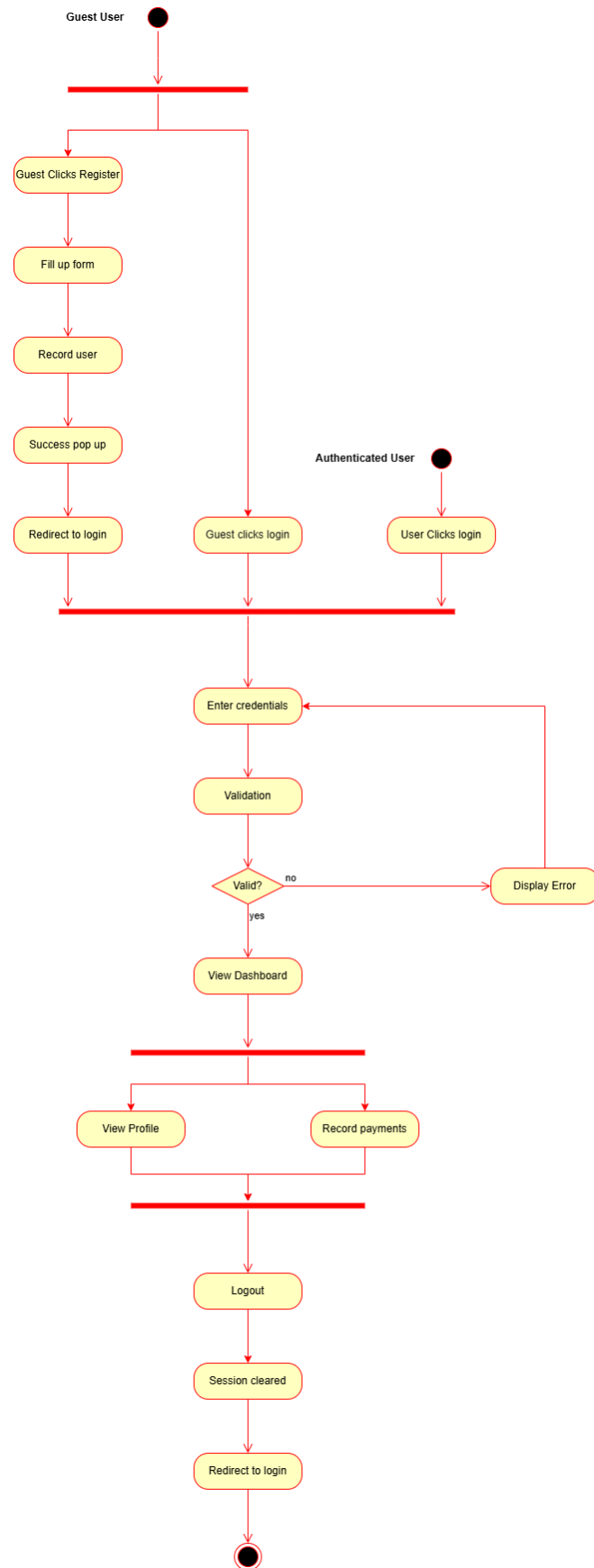
5.1. ERD



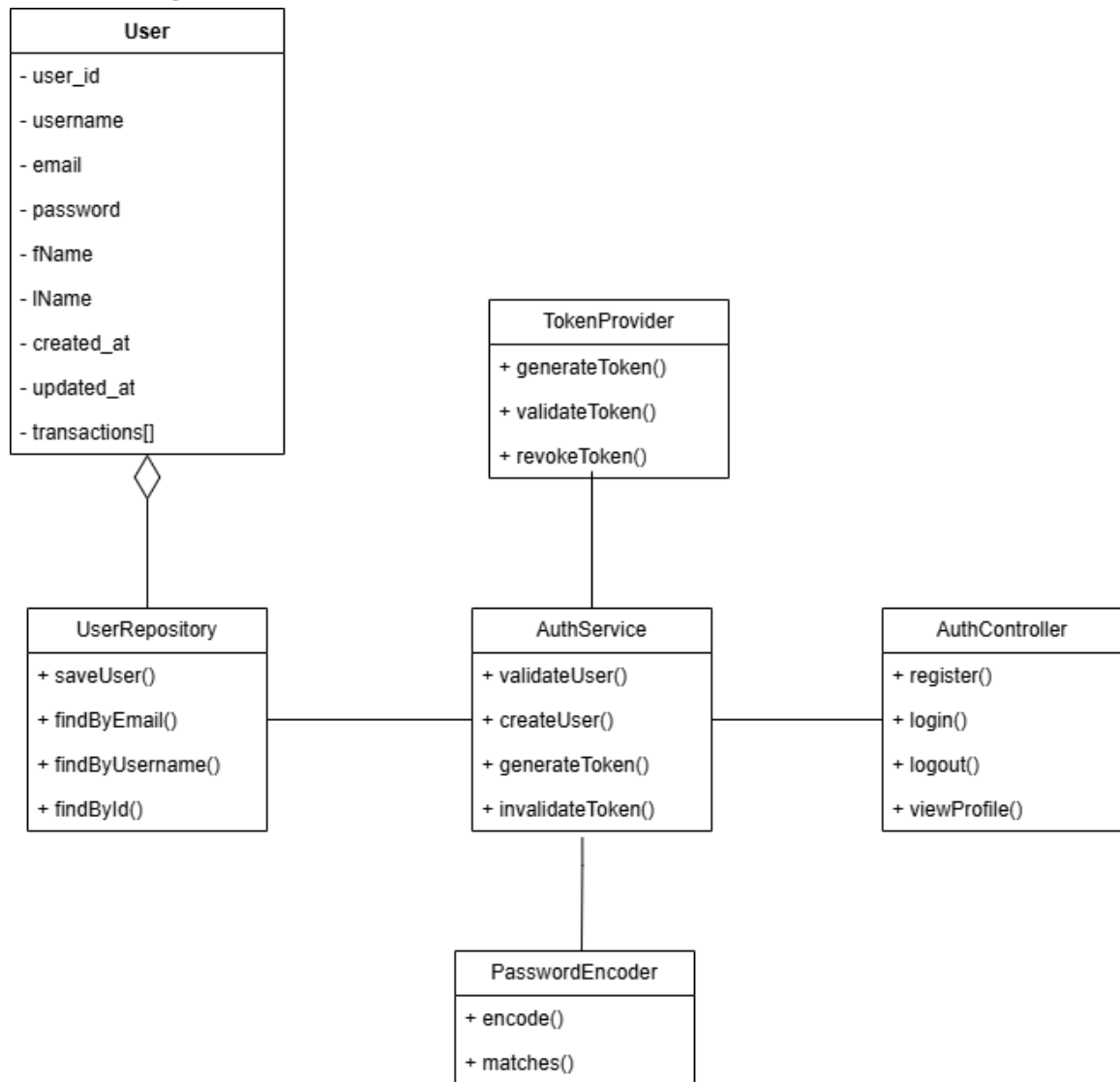
5.2. Use Case Diagram



5.3. Activity Diagram

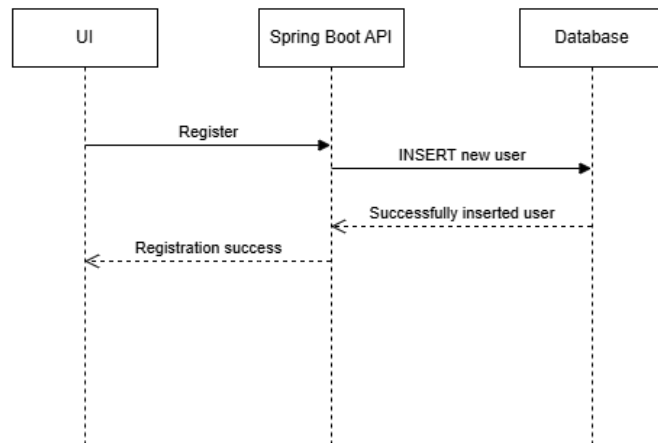


5.4. Class Diagram

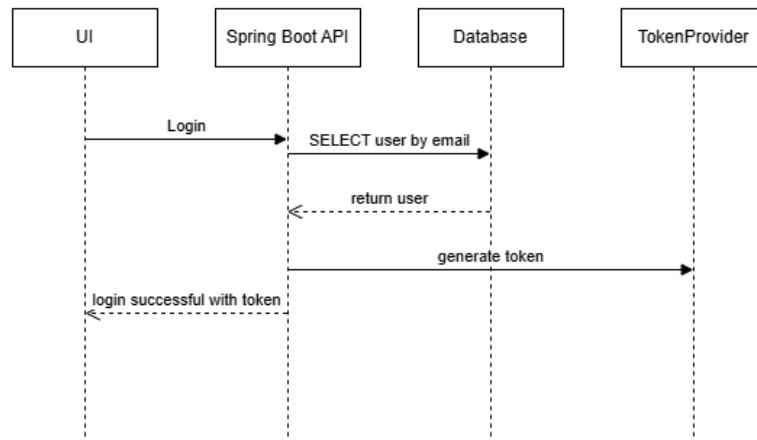


5.5. Sequence Diagram

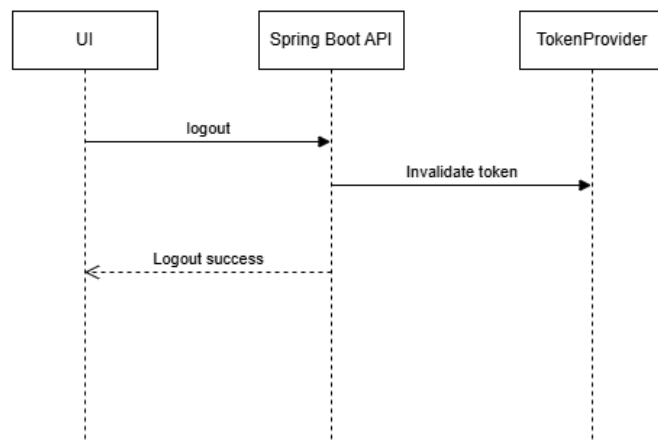
Register



Login



Logout



6. Appendices

Sample API Endpoints:

- POST /register → Create new user.
- POST /login → Authenticate user and return token.
- GET /dashboard → Return summary of transactions.
- POST /transaction → Add new transaction.
- PUT /transaction/:id → Update transaction.
- DELETE /transaction/:id → Delete transaction.
- POST /logout → Invalidate session token.